Data Science and Artificial Intelligence Program
DSAI 427 - Big Data Analytics - Spring 2026

Mini Project 1
The Digital Librarian (Distributed Reverse Indexing)

Doha Hemdan 202200701
Salma Sherif 202200622

1. Introduction / Problem Definition

Nowadays, traditional data processing methods have become severe bottlenecks. Relying on sequentially scanning documents (such as using the **grep** command) to find keywords does not scale; as a digital library expands from megabytes to terabytes, the latency for simple keyword searches becomes entirely unacceptable.

To solve this scaling issue, modern search engines rely on a Reverse Index. Instead of mapping a document to the words it contains, a reverse index maps a specific word to a list of the documents in which it appears, alongside its frequency in each document. This distributed approach enables fast information retrieval, benefiting anyone who relies on fast and accurate queries, such as librarians, academic researchers, and general search engine users.

2. Data Science Lifecycle Implementation

a. Storage Analysis: Discuss HDFS block storage, replication, and data locality concerning your chosen dataset.
   i. HDFS splits the data into blocks of 128 MB, but our data is very small like 400 KB, 700KB or even 1MB so nearly all the books are not achieving like 15MB so they are stored each book in one separate block. It seems that this is consuming the disk but it is taking its actual space from the disk like only the 15MB but for processing each book is in a separate block.
   ii. In the run.sh we have specified the replication of 1, and actually that in the docker-compose we have specified 2 data-nodes in a time, and 4 data-nodes in another time and nearly each one is having a copy of all the data for handling the fault tolerance factor which is also kind of replication.
   iii. Data Locality is achieved implicitly through YARN that each process is done beside the actual storage address of the data.

b. Preparation Impact: Analyze how stop-word removal reduced the amount of intermediate data (map output) and how this, in turn, lessened the burden of the "Shuffle" phase (network I/O).
   i. Removing stopwords will lower the burden on the Reducer, Combiner, and the Shuffle and sort. In addition the writing from and to the disk is lowered. However, the actual execution time is increased very slightly and this we think because we calculate the mapper time also so it takes some time to remove the stop words also we see this difference because this is a small data, but in large ones this difference will appear.

c. Quantify this if possible (e.g., "Stop-word removal filtered out approximately X% of tokens,reducing shuffle size by Y MB").
    i. Before Removing Stopwords: Map output records=429897

       Before Removing Stopwords: Map output bytes=7391019

    ii. After Removing Stopwords: Map output records=334841

        After Removing Stopwords: Map output bytes=5898919

    iii. Stop-word removal filtered out approximately 32% of tokens,reducing shuffle size by 1.4 MB
d. Processing Logic: Briefly explain your Mapper, Reducer, and (if implemented) Combiner logic.
    i. Mapper: It uses regular expressions (re.findall(r'[a-z0-9]+')) to extract alphanumeric tokens and normalizes them to lowercase. To improve the quality of the index, it checks each token against a predefined set of stop words (if the REMOVE_STOPWORDS flag is enabled) to filter out uninformative words. The remaining tokens are passed through NLTK's PorterStemmer to reduce words to their root form. t emits an intermediate key-value pair formatted as word,doc_id \t 1.
    ii. Combiner: It reads the intermediate outputs from the mapper and aggregates the counts for identical word,doc_id keys. nstead of transmitting thousands of individual (word,doc_id \t 1) pairs across the cluster, the combiner condenses them into a single pair with a summed frequency (e.g., word,doc_id \t 500). This drastically reduces the volume of data transferred during the shuffle phase, saving significant network bandwidth and memory.
    iii. Reducer: It processes the sorted stream of key-value pairs, keeping a running total of the frequencies for each specific word within each document. Once the total counts are tallied, it reshapes the data structure. It groups the output strictly by the unique word, attaching a dictionary or list of all documents containing that word along with their final term frequencies. he script prints the finalized inverted index format: word \t doc1:count, doc2:count, doc3:count.

3. Scalability Analysis & Results

3.1. Methodology:

3.1.1. Dataset size: 2.6 MB

3.1.2. Cluster configuration:

To measure performance scaling, the exact same MapReduce job was executed across three distinct cluster configurations:

- **Configuration 1 (1 Node):** Pseudo-distributed mode using a single DataNode and NodeManager.
- **Configuration 2 (2 Nodes):** A small cluster utilizing two DataNodes and two NodeManagers.
- **Configuration 3 (4 Nodes):** A larger cluster scaled up to utilize four DataNodes and four NodeManagers.

3.1.3. Metrics Recording:

Performance was measured by capturing the total Execution Time (T), defined as the total seconds from job submission to completion.

- **Automated Tracking:** This was recorded programmatically via a master execution script **run_project.sh**. The script captures a UNIX epoch timestamp **START_TIME=date** immediately before initiating the HDFS cleanup and Hadoop Streaming job, and a second timestamp **END_TIME=date** upon completion. The difference between these timestamps yields the exact execution time in seconds.
- **Speedup Calculation:** Using the recorded execution times, the Speedup (S) metric for each configuration was calculated using the formula **S=T_1/T_n**, where **T_1** is the baseline execution time on 1 node and **T_n** is the execution time on n nodes.

3.2. Performance Visualization:

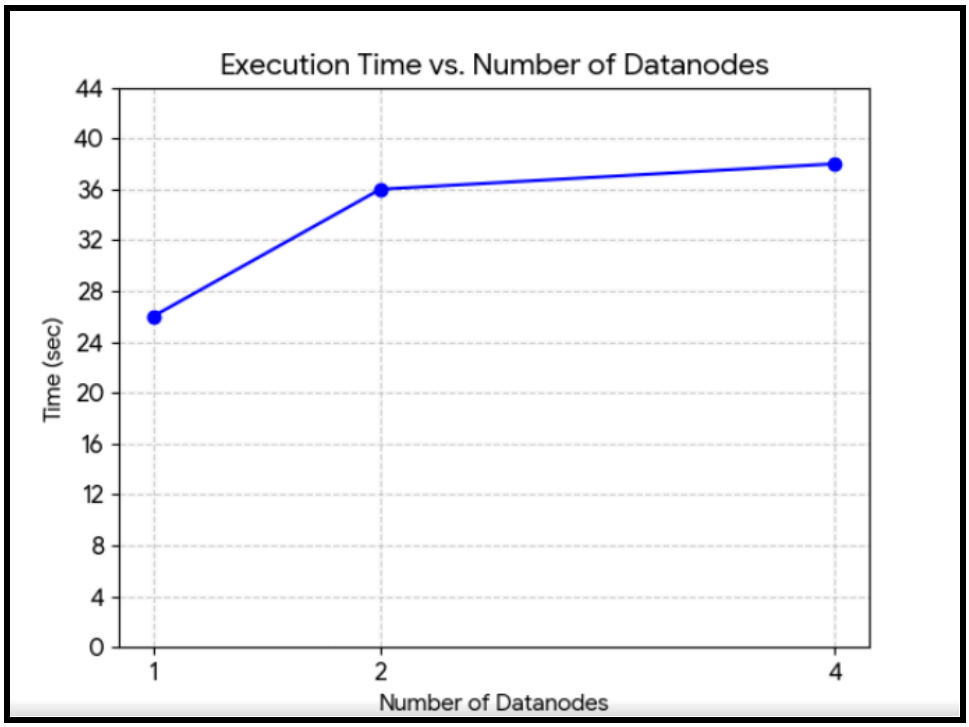

Figure1: Execution time comparison with different cluster configurations with 11 mapper and 1 reducer

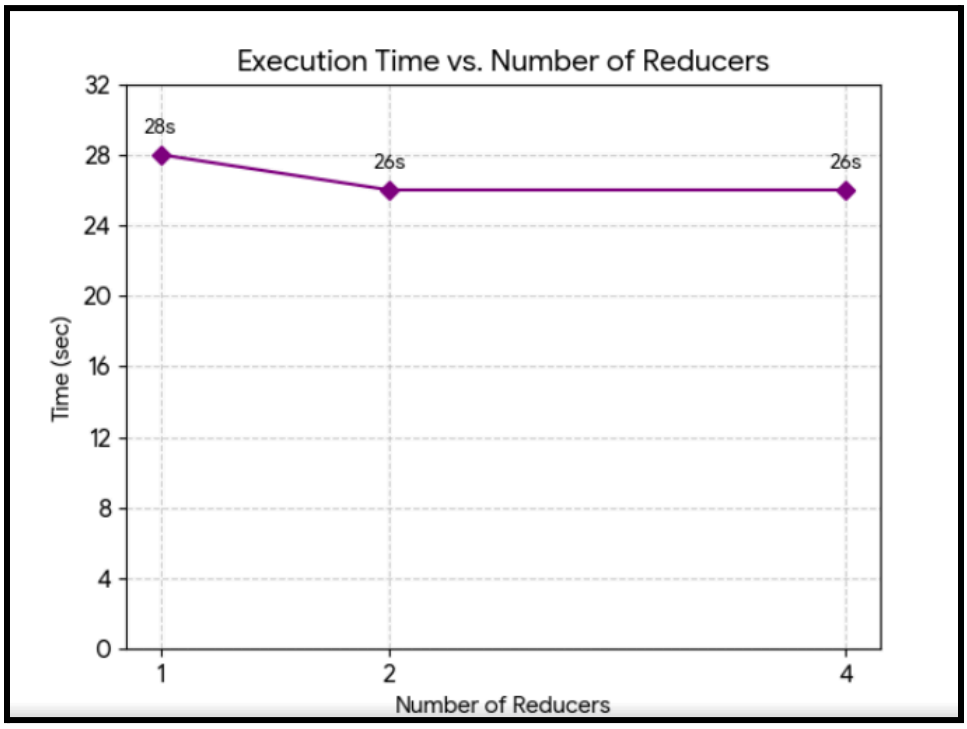

Figure2: Execution time comparison with different cluster configurations with 11 mapper and 2 reducer

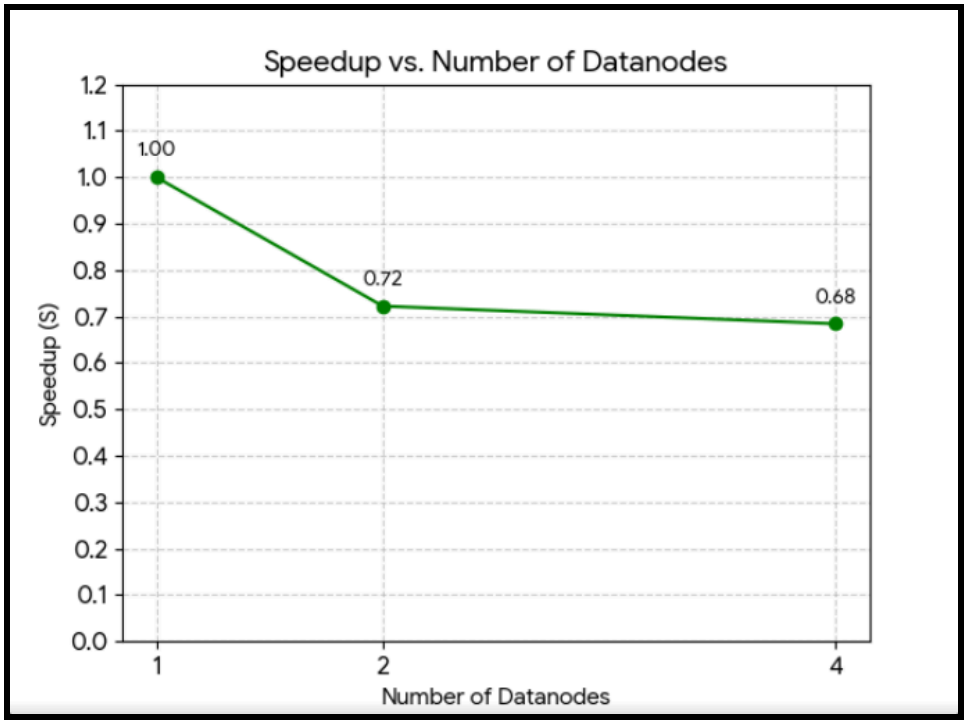

Figure3: Speedup comparison with different cluster configurations with 11 mapper and 1 reducer

3.3. Results Discussion

3.3.1. Did you achieve linear speedup?

linear speedup was not achieved. In fact, the experiment demonstrated a **negative (or degrading) speedup**. Instead of the execution time halving as the number of nodes doubled, the processing time actually *increased* from **26 seconds on a single node** to **38 seconds on four nodes.**

3.3.2. Analysis of Overheads & Amdahl's Law:

This result is common when testing Big Data frameworks on small datasets (like 10-20 text files) and can be attributed to several significant overheads:

- **Amdahl's Law and the Sequential Portion:** Amdahl's Law dictates that the maximum speedup of a system is **limited by the sequential portion of the task.** In this Hadoop job, the sequential overhead includes job submission, JVM instantiation, resource negotiation by the ResourceManager, and the orchestration of the MapReduce phases. Because the dataset is small, the **actual parallel processing time takes only a fraction of a second**, making the fixed sequential overhead dominate the total 38 seconds.
- **Task Initialization:** When scaling from 1 to 4 nodes, Hadoop must:
    - Negotiate with multiple NodeManagers.
    - Allocate containers

    - Initialize multiple Python processes across the cluster.

The administrative cost of **dividing a tiny workload into smaller chunks** and spinning up resources to handle them **outweighs the benefits of parallel execution**.

- **Network Communication (The Shuffle Phase):** In a 1-node configuration, intermediate data (map outputs) never leaves the local machine. Once 2 or 4 nodes are introduced, the framework must physically stream intermediate data across the network during the Shuffle & Sort phase so the Reducer can aggregate it. **This network I/O introduces a massive latency penalty** compared to local memory/disk aggregation.

3.4. Potential Bottlenecks:

- **Single-Host Resource Contention:** Because the cluster is **simulated** using Docker containers on a single physical host machine, adding more **"DataNodes" does not add more physical CPU, memory, or disk spindles.** Instead, it forces the host OS to rapidly context-switch between competing Docker containers **sharing the same physical resources, creating artificial CPU and Disk I/O bottlenecks.**
- **Dataset Size vs. Framework Overhead:** Hadoop is designed for datasets in the terabyte or petabyte range. For a few megabytes of text, the framework itself becomes the bottleneck. The overhead of the Hadoop streaming API, Python wrappers, and cluster coordination severely bottlenecked the job, proving that distributed computing introduces a performance penalty when applied to datasets that could easily fit into the RAM of a single machine.
-

4. Conclusion

This project successfully implemented a distributed inverted index using Hadoop MapReduce. Our pipeline worked exactly as intended, and by adding a Combiner and removing stop-words, we successfully optimized the workload—filtering out about 32% of the total tokens and reducing the network shuffle size by 1.4 MB.

However, our scalability testing resulted in a negative speedup. Instead of processing faster on more nodes, the execution time actually increased from 26 seconds on a single node to 38 seconds on a four-node cluster. This happened because our dataset was extremely small (only 2.6 MB).

For a dataset this tiny, Hadoop's fixed overhead takes significantly longer than the actual text processing. The time it takes YARN to negotiate resources, allocate containers, initialize JVMs, and physically stream intermediate data across the network during the Shuffle phase vastly outweighs the benefits of parallel processing. Ultimately, this experiment proved that while our MapReduce logic is correct and scalable, distributed frameworks like Hadoop are only beneficial when the dataset is massive enough to justify the setup overhead.